



The National University of Computer and
Emerging Sciences

Operator Overloading

Object-Oriented Programming (CS-1004)

Muhammad Zulqarnain

Department of Computer Science

Lecture slide credit goes to Dr. Akhtar Jamil from
the National University of Computer and
Emerging Sciences, Islamabad
(Email: akhtar.jamil@nu.edu.pk).

Goals

- Today's Lecture
 - Operator Overloading

Operator Overloading

- The method of *defining additional meanings for operators* is known as operator overloading
- Enables an operator to perform different operations depending upon the type of operands
- Basic operators i.e. +, -, *, / normally work with basic types i.e. double, float, int, long. (defined in C++)
- So, *how can these operators be applied to user-defined data types?*

Operator Overloading Motivation

- Don't want to create new operators for user-defined data types
- **Operator overloading:**
 - Enabling C++'s operators to work with class objects
 - Using traditional operators with user-defined objects
 - Requires great care; when overloading is misused, program difficult to understand
 - Examples of **already overloaded operators:**
 - Operator << is both the **stream-insertion operator** and the **bitwise left-shift operator**

Operator Overloading

- Example (already overloaded operator):

type `int` /type `int`

9 / 5

operator performs
int division

type `long` /type `long`

9L / 5L

operator performs
long division

type `double` /type `double`

9.0 / 5.0

operator performs
double division

type `float` /type `float`

9.0f / 5.0f

operator performs
float division

How to Overload an Operator?

- *An operator can be overloaded by declaring a special member function in the class*
- Name of the member function is operator that is followed by operator symbol e.g., **operator+**, **operator/**, etc.
- Can be an independent function (*except for the following operators: (), [], -> or **any of the assignment operators***)
- Can be a class's member function (**must be non-static**)

How to Overload an Operator?

- **Member function:** If the left operand of that particular class is an object of the same class, then the overloaded operator is said to be implemented by a member function.
- **Non-member function:** If the left operand of that particular class is an object of a different class, then the overloaded operator is said to be implemented by a non-member function.

Overload as Member or Non-Member Function

- If it is a ***unary operator***, implement it as a member function.
- If a ***binary operator*** treats both operands equally (it leaves them unchanged), implement this operator as a non-member function.
- If a ***binary operator*** does not treat both of its operands equally (usually it will change its left operand), it might be useful to make it a member function of its left operand's type.

Syntax to Overload an Operator

returnType operator opsymbol(parameters){ }

↑
any type

↑
keyword

↑
operator symbol

↑
function body

- **return-type** may be whatever the operator returns
- **Operator symbol** may be any overloadable operator

Example:

void operator+ (parameters){ }

↑
any type

↑
keyword

↑
operator symbol

↑
function body

Operator Overloading

- Operators are really just functions
 - They have arguments, they return values
 - The only difference is that their names take on a specific form:
Operator+, **operator[]**

- **Overloading provides concise notation:**

// without operator overloading

```
object2 = object1.add(object2) ;
```

// with operator overloading

```
object2 = object2 + object1;
```

Restriction on Operator Overloading

- With operator overloading **we cannot change**:
 1. **How operators act on built-in data types:**
 - i.e., cannot change integer addition
 2. **Precedence of operator (order of evaluation)**
 - Use parentheses to force order-of-operations
 3. **Association rules** (*left-to-right* or *right-to-left* evaluation)
 4. **Number of operands**
 - i.e., & is unary, only acts on one operand
 5. **Cannot create new operators**
 6. **Operators must be overloaded explicitly:**
 - i.e., Overloading + , does not overload +=

Restriction on Operator Overloading

Operators that can be overloaded							
+	-	*	/	%	^	&	
~	!	=	<	>	+=	--	*=
/=	%=	^=	&=	=	<<	>>	>>=
<<=	==	!=	<=	>=	&&		++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

http://www.stroustrup.com/bs_faq2.html#overload-dot

Operators that cannot be overloaded				
.	.*	::	?:	sizeof

Operator =, operator &

- **Operator =** and **Operator &** are overloaded implicitly for every class, so they can be used for each class object.
- **Operator =** performs member-wise copy of the data members.
- **Operator &** returns the address of the object in memory.

Function Overloading

- An overloaded function is one which has the same name but several different forms.
- **For example:** we can overload the constructor for the Date class:

<i>default</i>	Date d;
<i>initializing</i>	Date d(9,22,20);
<i>copy</i>	Date d1(d);
<i>other</i>	Date d("Sept",22,2020);

Operator Overloading

- The operator “+” also has different semantics depending on the type of its arguments.

- Example

```
int i, j;
```

```
double d, e;
```

```
i + j;    //add two int
```

```
i + d;    //add an int and a double
```


Operator Overloading Syntax

a = b + c;

datatype operator + (datatype) { ... }

Second parameter (can be native data type or user defined data type)

Remember **operator+** is a function, and it will be called with the help of any object, thus the first parameter is the calling object

return parameter

(can be native data type or user defined data type)

Operator Overloading Syntax

datatype operator+ (datatype) { }

Example (1):

```
class myClass
{
    int operator+ ( int );
}
```

```
int main ( )
{
    int a, b;
    myClass object;
    a = object + b;
}
```

Operator Overloading Syntax

datatype operator+ (datatype)

Example (2):

```
class myClass
{
    int operator+ ( myClass &a );
}

int main ( )
{
    int a;
    myClass object1, object2;
    a = object1 + object2;
}
```

Operator Overloading Syntax

datatype operator+ (datatype)

Example (3):

```
class myClass
{      myClass operator+ ( int );      }
```

```
int main ( )
{
    int a = 5;
    myClass object1, object2;
    object2 = object1 + a;
}
```

Operator Overloading Syntax

datatype operator+ (datatype)

Example (4):

```
class myClass
{
    myClass operator+ ( myClass &a );
}

int main ( )
{
    myClass object1, object2, object3;
    object3 = object1 + object2;
}
```

Implementing Overloaded Operators

- The compiler uses the types of arguments to choose the appropriate overloading.

```
int v1, v2;
```

```
v1 + v2; // int +
```

```
float s1, s2;
```

```
s1 + s2; // float+
```

Extended Example

Employee class and objects

```
class Employee
{
    private:
        int idNum;
        double salary;
    public:
        Employee(int id, double salary);
        double addTwo (Employee& emp);
        double operator+ (Employee& emp);
        double getSalary() { return salary; }
};
```

Member functions

'addTwo' and operator+

//function notation

```
double Employee::addTwo(Employee& emp)
{
    double total;
    total = this->salary + emp.getSalary();
    return total;
}
```

//operator overloading notation

```
double Employee::operator+(Employee& emp)
{
    double total;
    total = this->salary + emp.getSalary();
    return total;
}
```


Member functions

'addTwo' and operator+

```
double sum;
```

```
Employee Clerk (111, 10000), Driver (222, 6000);
```

```
// these three statements do the same thing
```

```
sum = Clerk.addTwo(Driver);
```

```
sum = Clerk.operator+(Driver);
```

```
sum = Clerk + Driver;
```

```
// the syntax for the last one is the most natural
```

```
// and is easy to remember because it is consistent
```

```
// with how the + operator works for everything else
```

Multiple Operators

- Often, you may need to reference an operator more than once in an expression:

Example:

```
total = a + b + c;
```

- **But this can cause big problems when operator overloading is involved**

Client Code for Class Employee

```
void main()
{
    Employee Clerk(115, 20000.00);
    Employee Driver(256, 15500.55);
    Employee Secretary(567, 34200.00);
    double sum;

    sum = Clerk + Driver + Secretary;

    cout << "Sum is " << sum;
}
```

The Problem

- **Operator +** is left to right associative, so Clerk and Driver are added. The result is a double.
- Now the resulting double is on the left and an Employee is on the right (i.e., *Secretary*).
- BUT THE OPERATOR + is only defined for arguments of type Employee, **not for double.**

The Problem Gets Worse

- It would seem that all we have to do is write another version of the overloaded operator to work with the argument (double).
- **But**
 - although we could overload an operator to work like this:
`sum = Secretary + num;`
- We cannot overload one like this:
`sum = num + Secretary; // why not?`

The Answer

- We cannot overload + for a double (a native type)
- The **real solution** is to make sure that your operator+ function never returns a double (or any other native type).
- An operator to add Employees should return an Employee (see next slide)

Extended Example

Employee class and objects

```
class Employee
{
    private:
        int idNum;
        double salary;
    public:
        Employee(int id, double salary);
        Employee operator+ (Employee& emp);
        double getSalary() { return salary; }
}
```

Solution Example

```
Employee Employee::operator+(Employee& emp)
{
    Employee total(999,0); // dummy values
    total.salary = salary + emp.salary;
    return(total);
}
```


Client Code for Class Employee

```
void main()
{
    Employee Clerk(115, 20000.00);
    Employee Driver(256, 15500.55);
    Employee Secretary(567, 34200.00);
    Employee sum(0, 0.0);
    sum = Clerk + Driver + Secretary;
}
```

Invoking Objects

- If the operator is binary but there is only one explicit argument, the '*invoking instance*' is assumed to be the one on the *left hand side* of the expression.

Rectangle operator+(Rectangle& r)

```
{  
    Rectangle temp;  
    temp.x = this->x + r.x;  
    temp.y = this->y + r.y;  
    temp.w = this->w + r.w;  
    temp.h = this->h + r.h;  
    return temp;  
}
```

void main()

```
{  
    Rectangle r3 = r1 + r2;  
}
```

The Answer (double+object)

- We cannot overload + for a double (a native type)
- The **real solution** is to make sure that your operator+ function never returns a double (or any other native type).
- An operator to add Employees should return an Employee (see next slide)
- **Solution 2: make a non-member operator overloading function.**

```

class myClass
{
    private:
        int x;
    public:
        myClass(int x=0) { this->x=x; }

        //Getter function
        int getX(){
            return x;
        }

        //Setter function
        void setX(int x){
            this->x=x;
        }
};

myClass operator+ (myClass &a, myClass &b)
{
    myClass temp;
    temp.setX(a.getX()+b.getX());
    return temp;
}

int main ( )
{
    myClass object1,object2,object3;
    object1.setX(10);
    object2.setX(5);
    object3 = object1+object2;
    cout<<object3.getX();
}

```

Non-member Operator Overloading Function

Assignment Operator =

- **Operator =** is overloaded implicitly for every class, so it can be used for all class objects.
- Operator = performs member-wise copy of the data members.
- However, there is a problem with implicitly overloaded operator...(see next slide)

Using Implicitly Overloaded Assignment Operator

```
// A class without user defined assignment operator
class Test
{
    int *ptr;
public:
    Test (int i = 0)    { ptr = new int(i); }
    void setValue (int i) { *ptr = i; }
    void print()        { cout << *ptr << endl; }
};

int main()
{
    Test t1(5);
    Test t2;
    t2 = t1;
    t1.setValue(10);
    t2.print();
    return 0;
}
```

Practice Tasks

```
class vector
```

```
{
```

```
public:
```

```
private:
```

```
    float* data;
```

```
};
```

- Update the above class and add required members (if needed).
- data is a pointer to an array
- Implement all possible operators that can be overloaded for different scenarios.
- e.g == can be used to check if two vectors are same, <= to check if element wise the elements of one vector are less than the others.

References

- Chapter 14, Starting Out with C++ - From Control Structures through Objects-Pearson (2019)

Thank You 😊